

Portfolio Optimisation with a Genetic Algorithm

leon5354

2026-07-19

Contents

Introduction	1
Data	1
Part 1: optimising weights on a fixed basket	2
Part 2: selecting stocks with a binary GA	5
Discussion	6
Reproducibility	7

Introduction

This report explores portfolio weight optimisation using a genetic algorithm (GA). Given a universe of stocks, the goal is to decide what fraction of the portfolio to allocate to each stock so that an objective (Sharpe ratio, raw return, or risk) is optimised, subject to the constraint that the weights sum to one and each individual weight lies in $[0, 1]$.

The analysis is split into two parts:

- **Part 1** — start with a hand-picked 10-stock basket and compare four fitness functions (max-Sharpe, max-return, min-risk, and two scaled Sharpe variants).
- **Part 2** — start from a 50-stock universe and use a binary GA to pick the 10 best stocks by Sharpe ratio, then re-optimize weights.

Portfolios trained on the 2020–2021 period are then re-evaluated on the out-of-sample 2021–2022 period as a simple sanity check.

Disclaimer: This is a learning exercise. Nothing here is investment advice, and the “validation” approach is far too simplistic to support any real trading decision.

Data

Daily returns are loaded from a snapshot cached in `data/cache/` (see `data/fetch_data.R` for how to refresh it). As of 2024, the `quantmod::getSymbols` Yahoo Finance path no longer works, so the snapshots are committed to keep the report runnable.

```
suppressMessages({ library(GA); library(xts); library(ggplot2); library(scales);
                   library(forcats); library(tidyr); library(dplyr) })

cache <- "data/cache"
myRetData <- readRDS(file.path(cache, "returns_10_2020_2021.rds"))
myRetData <- myRetData[-1, ] # drop the first all-NA row from diff()
myRetData_future <- readRDS(file.path(cache, "returns_10_2021_2022.rds"))[-1, ]
myRetData_fif <- readRDS(file.path(cache, "returns_50_2020_2021.rds"))[-1, ]
```

The Part-1 basket covers a mix of ETFs, large-cap tech, retail, and crypto-equity exposure:

```
## QQQ, VOO, AAPL, HRB, AMZN, ETHE, GBTC, ZM, NVDA
```

A few tickers from the original project (notably JWN, DISCA, SQ) are no longer available through the Yahoo free API and are dropped automatically.

Part 1: optimising weights on a fixed basket

Constraint and objective

Because the GA package performs maximisation on a single objective, we collapse return and risk into a single fitness. For the maximum-Sharpe variant (with a 0% risk-free rate):

```
sharpe <- function(x) {
  mean(x * myRetData) / sqrt(t(x) %*% cov(myRetData) %*% x)
}
```

The full-investment and box constraints are enforced through a quadratic penalty rather than hard bounds, so the GA can search freely in $[0, 1]^n$:

```
all_weight <- function(x) {
  w <- (sum(x) - 1)^2
  for (i in seq_along(x)) {
    w <- w + max(c(0, x[i] - 1))^2 + max(c(0, -x[i]))^2
  }
  w
}
```

The fitness combines the two. `-sharpe` because GA maximises:

```
obj <- function(x) -sharpe(x) + 100 * all_weight(x)
```

Maximum-Sharpe portfolio

The GA converges with fitness 0.02142, giving the following weight allocation:

Table 1: Maximum-Sharpe portfolio weights

Stock	Weight
ETHE	0.2086
HRB	0.1654
AMZN	0.1440
VOO	0.1356
GBTC	0.1091
ZM	0.0788
AAPL	0.0728
QQQ	0.0564
NVDA	0.0294

Comparing fitness functions

To understand how the choice of objective shapes the portfolio, we run four GA variants:

1. **Max return** — ignore risk, maximise mean return.
2. **Min risk** — minimise portfolio volatility.
3. **More return** — Sharpe ratio with risk scaled down by 0.5 (rewards return).
4. **Less risky** — Sharpe ratio with risk scaled up by 1.5 (penalises risk).

Table 2: Portfolio weights under each objective

stock	max_sharpe	max_return	min_risk	more_return	less_risky
QQQ	0.056	0.059	0.145	0.075	0.109
VOO	0.136	0.091	0.119	0.132	0.085
AAPL	0.073	0.136	0.116	0.075	0.062
HRB	0.165	0.115	0.119	0.160	0.172
AMZN	0.144	0.122	0.166	0.137	0.149
ETHE	0.209	0.153	0.036	0.168	0.172
GBTC	0.109	0.113	0.079	0.103	0.042
ZM	0.079	0.120	0.134	0.085	0.137
NVDA	0.029	0.090	0.085	0.066	0.073

As expected, the **max-return** portfolio concentrates on the highest-mean stocks, the **min-risk** portfolio spreads weight more evenly, and the scaled Sharpe variants sit in between.

Out-of-sample sanity check

We compare each portfolio's Sharpe ratio on the 2020–2021 training period against its Sharpe ratio on the held-out 2021–2022 period.

```
allratio <- function(x, data) {
  port_returns <- mean(x * data)
  port_risk    <- sqrt(t(x) %*% cov(data) %*% x)
  c(sharpe = port_returns / port_risk,
    risk   = port_risk,
    return = port_returns)
```

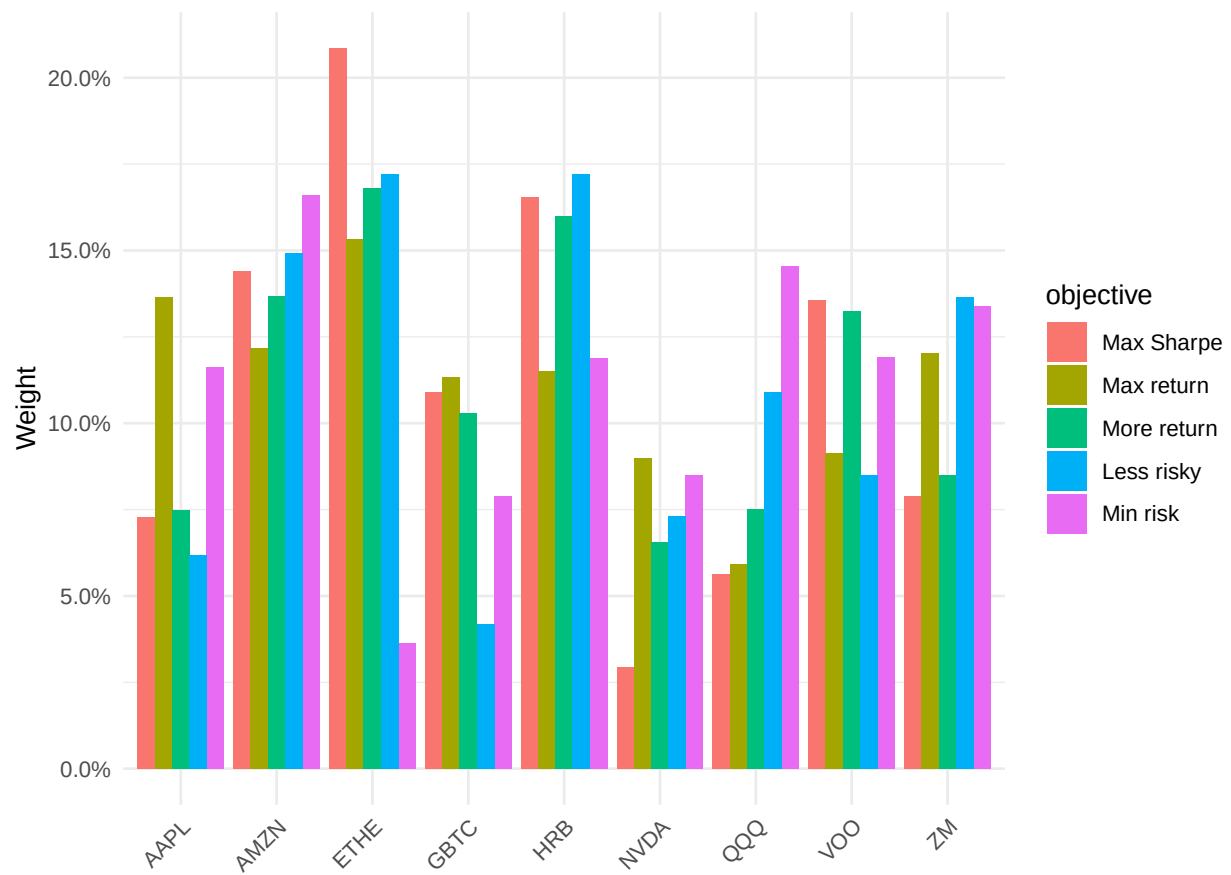


Figure 1: Optimal weights under each objective variant.

```
}  
  
weight_equal <- rep(1 / ncol(myRetData), ncol(myRetData))
```

Table 3: In-sample (2020-2021) vs out-of-sample (2021-2022) performance

portfolio	sharpe_train	risk_train	return_train	sharpe_test	risk_test	return_test
Max Sharpe	0.021	0.036	0.001	0.004	0.021	0
Max return	0.014	0.032	0.000	0.007	0.020	0
Min risk	0.010	0.024	0.000	0.011	0.014	0
More return	0.020	0.033	0.001	0.006	0.019	0
Less risky	0.019	0.032	0.001	0.007	0.018	0
Equal weight	0.013	0.029	0.000	0.008	0.018	0

Unsurprisingly, the best in-sample Sharpe does **not** translate into the best out-of-sample Sharpe — 2020 had an unusual combination of strong momentum and low correlation that simply did not persist. The min-risk portfolio degrades the least, consistent with the usual finding that volatility is more persistent than mean return.

Part 2: selecting stocks with a binary GA

Instead of hand-picking 10 stocks, we use a binary GA to pick 10 out of a 50-stock universe. Each bit indicates whether a stock is included; the fitness is the sum of Sharpe ratios of the selected stocks, subject to a cardinality constraint of exactly 10.

The binary GA selects the following 10 stocks from the 50-name universe (ranked by their individual Sharpe ratios):

Table 4: Top-10 stocks chosen by the binary GA

Selected stock	Individual Sharpe
RIOT	0.013
ZM	0.007
SNAP	0.005
ROKU	0.004
SIG	0.004
ARKK	0.004
NVDA	0.003
AMD	0.003
TSM	0.003
QQQ	0.002
MSFT	0.002
NKE	0.001
EA	0.001
GOOG	0.001
MCD	0.001
IBM	0.000
PFE	0.000
DAL	0.000

Selected stock	Individual Sharpe
MANU	0.000
AMC	-0.001

We then re-optimize the weights of those 10 stocks using the same max-Sharpe GA from Part 1.

Table 5: Optimised weights for the GA-selected top 10

Stock	Weight
ARKK	0.1040
NKE	0.1039
EA	0.1013
SIG	0.0916
ROKU	0.0909
MSFT	0.0889
NVDA	0.0879
AMD	0.0879
GOOG	0.0755
RIOT	0.0637
MANU	0.0487
IBM	0.0481
TSM	0.0480
PFE	0.0438
MCD	0.0436
DAL	0.0418
AMC	0.0413
ZM	0.0336
QQQ	0.0282
SNAP	0.0262

Because the 10 were chosen specifically for their historical Sharpe ratios, the in-sample Sharpe of this portfolio is higher than the hand-picked Part-1 portfolio. Out-of-sample, however, it suffers from the same 2020-specific regime change — possibly more so, since picking “winners” from one period is a classic source of overfitting.

Discussion

A few takeaways from the exercise:

- The genetic algorithm works fine for portfolio weight optimisation given a sensible penalty-based constraint encoding. It is not the most efficient tool — convex optimisation (e.g. the **quadprog** package) would solve the min-variance and max-Sharpe problems exactly and quickly — but it is flexible and makes the trade-off between objectives easy to explore.
- The choice of objective dominates everything else. Max-return, min-risk, and Sharpe variants produce visibly different portfolios from the same universe.
- Out-of-sample, none of the in-sample optima survive intact. The min-risk portfolio degrades the least, which is consistent with the broader empirical finding that covariance is more persistent than mean returns.

Reproducibility

Refresh the data with `Rscript data/fetch_data.R`. Re-render the report with `rmarkdown::render("report/portfolio-ga`

```
## R version: x86_64-apple-darwin17.0 (64-bit)
```

```
## Packages loaded: GA, xts, ggplot2
```